

=====

Agile/Scrum سیستم مدیریت وظایف هوشمند مبتنی بر

نسخه 1.0 - مستندات کامل و تفصیلی

فهرست مطالب

بخش اول: معرفی و معماری

1. معرفی کلی پروژه

2. معماری و ساختار کلی سیستم

3. تکنولوژی‌ها و ابزارها

بخش دوم: موجودیت‌ها و مدل داده

4. (Core Entities) موجودیت‌های اصلی

5. (Collaboration Entities) موجودیت‌های همکاری

6. (Tracking Entities) موجودیت‌های ردیابی

7. روابط و ارتباطات بین موجودیت‌ها

بخش سوم: لایه‌های اپلیکیشن

8. (Controllers Layer) لایه کنترلرها

9. (Service Layer) لایه سرویس

10. (Data & Repository Layer) لایه دیتا و ریپازیتوری

11. Infrastructure و Background Jobs لایه

بخش چهارم: قابلیت‌ها و ویژگی‌ها

12. مدیریت کاربران و احراز هویت

13. Workspace مدیریت

مدیریت پروژه و اسپرینت 14.

15. Backlog و User Story مدیریت

16. Task مدیریت و زیرمجموعه‌ها

17. داشبوردها و گزارشات تحلیلی

18. (وابستگی‌ها، تحلیل‌گرها، AI) ویژگی‌های پیشرفته

بخش پنجم: پیکربندی و استقرار

19. پیکربندی‌های سیستم

20. امنیت و دسترسی‌ها

21. Quality Assurance تست‌ها و

22. راهنمای نصب و استقرار

23. نگهداری و عیب‌یابی

بخش ششم: مراجع

24. Enumها و ثابت‌ها

25. DTOها و ViewModel

26. واژمنامه اصطلاحات

27. (FAQ) سوالات متداول

28. Roadmap و ویژگی‌های آینده

=====

بخش اول: معرفی و معماری

=====

معرفی کلی پروژه ۱.

(Overview) نگاه کلی

و ASP.NET Core 8.0 یک سیستم مدیریت وظایف و پروژه تحت وب است که بر اساس معماری SmartTask طراحی و پیاده‌سازی شده است. این سیستم یک راهکار یکپارچه و جامع برای مدیریت چرخه Agile/Scrum الگوی حیات کامل پروژه‌های نرم‌افزاری فراهم می‌کند - از برنامه‌ریزی اولیه تا تحویل نهایی

****ویژگی‌های کلیدی****

- Scrum (Sprint, Backlog, User Story, Task) پشتیبانی کامل از فریم‌ورک
- SignalR با Real-time ارتباطات
- داشبوردهای تحلیلی و گزارش‌های پیشرفته
- ابزارهای همکاری تیمی (نظرات، مثنی، نوتیفیکیشن)
- و تحلیل پروژه Task هوش مصنوعی برای تجزیه
- مدیریت زمان و ردیابی پیشرفت
- سیستم پیچیده دسترسی و نقش‌ها
- قابلیت مقیاس‌پذیری برای تیم‌های کوچک تا بزرگ

هدف و چشم‌انداز پروژه ۱.۱.

****هدف اصلی****

ارائه پلتفرمی یکپارچه، قدرتمند و کاربرپسند برای تیم‌های نرم‌افزاری جهت برنامه‌ریزی، پیگیری و مدیریت پروژه‌ها با و افزایش بهره‌وری تیم (Agile) رویکرد چابک

****اهداف فرعی****

- کاهش پیچیدگی فرآیندهای مدیریت پروژه
- در پروژه‌ها (Visibility) افزایش شفافیت و قابلیت دید
- بهبود همکاری و ارتباط اعضای تیم
- ارائه تحلیل‌های داده‌محور برای تصمیم‌گیری بهتر
- خودکارسازی کارهای تکراری و وقت‌گیر
- (Scrum, Kanban) پشتیبانی از روش‌های مختلف مدیریت پروژه

۱.۲. Agile و Scrum معرفی

****Agile (چابک)****

همکاری تیمی، بازخورد مشتری و (Iterative) یک روش‌شناسی توسعه نرم‌افزار است که بر توسعه تکراری Agile بر تحویل مداوم نسخه‌های کاری Agile، انعطاف‌پذیری در برابر تغییرات تأکید دارد. به جای برنامه‌ریزی کامل در ابتدا، نرم‌افزار تمرکز دارد.

****Scrum****

است که شامل Agile یک فریم‌ورک عملیاتی در Scrum:

- بازه زمانی ثابت (معمولاً ۲-۴ هفته) برای تحویل قابلیت‌های کاری: ****Sprint****
- لیست اولویت‌دار نیازمندی‌ها: ****Product Backlog****
- جلسه برنامه‌ریزی اسپرینت: ****Sprint Planning****
- جلسات روزانه کوتاه: ****Daily Standup****
- بررسی نتایج اسپرینت: ****Sprint Review****
- بررسی فرآیند و بهبود مستمر: ****Sprint Retrospective****

را به صورت دیجیتال پیاده‌سازی کرده است Scrum تمامی مفاهیم و فرآیندهای SmartTask.

۱.۳. مخاطبان سیستم

****کاربران اصلی****

- ****Product Owner****: تعریف و اولویت‌بندی User Storyها
- ****Scrum Master / Project Manager****: هدایت تیم و مدیریت فرآیند
- ****Task پیاده‌سازی: (Developers)**** توسعه‌دهندگان
- ****QA/Tester****: تست و بررسی کیفیت تسترها
- ****Stakeholders****: مشاهده پیشرفت و گزارش‌ها ذی‌نفعان

****سازمان‌های هدف****

- تیم‌های توسعه نرم‌افزار (۳ تا ۱۰۰+ نفر)
- شرکت‌های نرم‌افزاری
- استارت‌آپ‌ها

- سازمان‌ها IT دیپارتمان‌های
- (Product Teams) تیم‌های محصول

۱.۴. Technology Stack (تکنولوژی‌های استفاده شده)

Backend Technologies

****فریم‌ورک اصلی****

- ****ASP.NET Core 8.0****: جدیدترین نسخه فریم‌ورک متن‌باز مایکروسافت
 - MVC (Model-View-Controller) معماری -
 - Cross-platform (Windows, Linux, macOS)
 - عملکرد بالا و مقیاس‌پذیری -
 - پشتیبانی بلندمدت (LTS) -

****ORM: دیتابیس و****

- ****SQL Server****: پایگاه داده رابطه‌ای قدرتمند
 - Transaction پشتیبانی از -
 - Indexing قابلیت‌های پیشرفته -
 - Query پیچیده عملکرد بالا در -
- ****Entity Framework Core 8.0****: ORM (Object-Relational Mapping)
 - Code-First Migration
 - LINQ Query
 - Change Tracking
 - Lazy/Eager Loading

****احراز هویت و امنیت****

- ****ASP.NET Core Identity****: سیستم مدیریت کاربران

- Hash (Password Hashing) امن رمز عبور

- (2FA) احراز هویت دو مرحله‌ای

- (Role Management) مدیریت نقش‌ها

- Token-based Authentication

• (آماده، غیرفعال) Google ورود با: **OAuth 2.0**

****Real-time Communication:****

• Real-time ارتباطات دو طرفه: ****SignalR****

- WebSocket Protocol

- Long Polling به Fallback

- Hub-based Architecture

- Group Management برای نوتیفیکیشن‌های هدفمند

Libraries و Packages

****Export گزارش‌گیری و****

• Excel تولید فایل‌های: ****ClosedXML (v0.105.1)****

- Workbook و Worksheet ایجاد

- فرمت‌بندی سلول‌ها

- نمودارها و جداول

• PDF تولید: ****QuestPDF (v2026.7.2)****

- Layout و مدرن Fluent API

- پشتیبانی از فارسی

- عملکرد بالا

****Testing:****

- فریم‌ورک تست: ****xUnit (v2.9.3)****
- Mock کردن Dependencies: ****Moq (v4.20.72)****
- واقعی SQL Server تست بدون نیاز به: ****InMemory Database****

****Background Processing:****

- ****IHostedService****: Background Services
 - ارسال خودکار یادآورها: **ReminderBackgroundService**
 - های سررسید گذشته Task آپدیت وضعیت: **OverdueCascadeBackgroundService**

Frontend Technologies

****UI Framework:****

- CSS فریم‌ورک: ****Bootstrap 5****
 - Responsive Grid System
 - کامپوننت‌های آماده -
 - Utility Classes
- JavaScript کتابخانه: ****jQuery****
 - DOM Manipulation
 - AJAX Requests
 - Event Handling

****Icons و Assets:****

- آیکون‌ها: ****Font Awesome****
- فونت فارسی: ****Vazirmatn****

****JavaScript Libraries:****

- نمودارها و گراف‌ها: ****Chart.js****

- ****SortableJS****: Drag & Drop
- ****SignalR Client****: اتصال به Hub

External Services و Integrations

****هوش مصنوعی****

- ****LM Studio****: محلی AI سرور
 - Model: Google Gemma 4-12B
 - Task و تجزیه Chat
 - (خارجی API بدون وابستگی به) Self-hosted

****ایمیل****

- ****SMTP Gmail****: ارسال ایمیل
 - Workspace دعوتنامه
 - یادآورها
 - نوتیفیکیشن‌های مهم

****Push Notification****

- ****Webpushr****: نوتیفیکیشن مرورگر
 - اطلاع‌رسانی خارج از برنامه
 - پشتیبانی از تمام مرورگرها

Development Tools

****IDE و Editor****

- Visual Studio 2022
- Visual Studio Code

- SQL Server Management Studio (SSMS)

****Version Control:****

- Git
- GitLab یا GitHub (احتمالاً)

****Package Managers:****

- .NET پکیج‌های NuGet
- (اگر نیاز باشد) Frontend پکیج‌های npm

Architecture Patterns

****الگوهای طراحی پیاده‌سازی شده:****

- انتزاع دسترسی به داده: ****Repository Pattern****
- مدیریت Transaction: ****Unit of Work****
- کاهش وابستگی و افزایش تست‌پذیری: ****Dependency Injection****
- جداسازی منطق کسب‌وکار: ****Service Layer Pattern****
- جداسازی View از Model: ****ViewModel Pattern****
- ViewState حذف منطقی با: ****Soft Delete Pattern****
- Real-time برای SignalR Hubs: ****Hub Pattern****

SmartTask مزایای رقابتی ۱.۵

**** (Jira, Trello, Asana) نسبت به ابزارهای مشابه:****

**** و مالکیت کامل داده Self-hosted  ****

- هیچ هزینه اشتراک ماهانه
- کنترل کامل بر داده‌ها
- بدون محدودیت کاربر

✓ **سفارشی‌سازی کامل**

- کد منبع در دسترس
- قابلیت تغییر و توسعه
- اضافه کردن ویژگی‌های سازمانی

✓ **پشتیبانی کامل از زبان فارسی**

- فارسی UI و UX
- تقویم شمسی
- منطقه زمانی ایران

✓ **هوش مصنوعی داخلی**

- های خارجی API بدون وابستگی به
- حفظ حریم خصوصی
- بدون هزینه اضافی

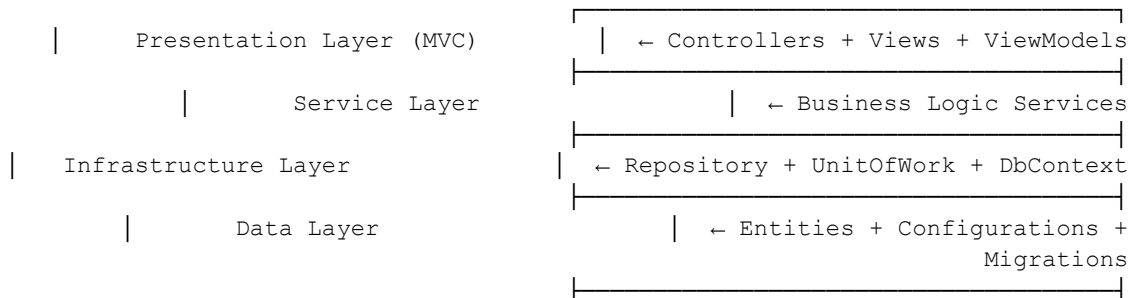
✓ **عدم نیاز به اینترنت (بعد از نصب)**

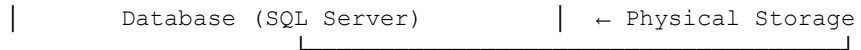
- قابل استفاده در شبکه محلی
- امنیت بالاتر
- سرعت بیشتر

معماری و ساختار کلی سیستم ۲.

(لایه‌ای) Layered معماری ۲.۱.

(چند لایه (طراحی شده که مسئولیت‌ها را به صورت منطقی جدا می‌کند N-Tier بر اساس معماری SmartTask

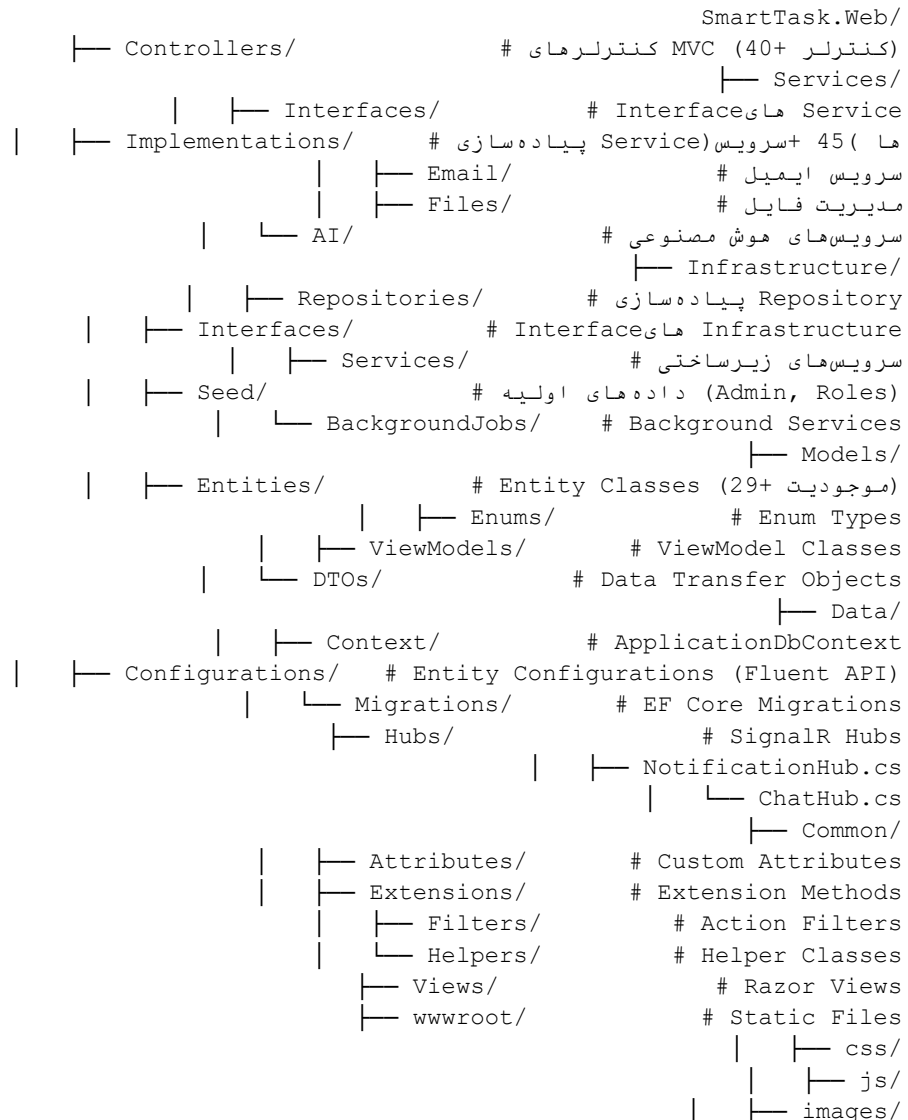




****مزایای این معماری****

- ****Separation of Concerns****: هر لایه یک مسئولیت مشخص
- ****Testability****: امکان تست مستقل هر لایه
- ****Maintainability****: تغییرات محدود به یک لایه
- ****Scalability****: قابلیت مقیاس‌پذیری آسان‌تر

۲.۲. ساختار پوشه‌ها (Project Structure)



****مزایا****

- Atomic Operations
- Transaction مدیریت متمرکز
- Partial Updates جلوگیری از

Dependency Injection

هدف ****بکاهش وابستگی و افزایش انعطاف‌پذیری****

ثبت می‌شوند `Program.cs` ها در Service تمام

```
builder.Services.AddScoped<IWorkspaceService, WorkspaceService>();
builder.Services.AddScoped<IProjectService, ProjectService>();
// ... 45+ سرویس دیگر
```

****مزایا****

- Loose Coupling
- تست‌پذیری بالا
- خودکار Lifecycle مدیریت

Service Layer Pattern

هدف ****جداسازی منطق کسب‌وکار از**** Repository و Controller

```
public class TaskService : BaseService<TaskItem>, ITaskService
{
    // Business logic here
    public async Task<bool> CanCompleteTaskAsync(int taskId, int userId)
    {
        ...بررسی وابستگی‌ها، دسترسی، وضعیت و ...
    }
}
```

****مزایا****

- HTTP Handling بر Controller تمرکز
- منطق کسب و کار Reusability
- تست آسان تر

Soft Delete Pattern

****هدف****: حذف منطقی به جای حذف فیزیکی رکوردها.

ارث می‌برند `BaseEntity` ها از Entity تمام

```
public abstract class BaseEntity
{
    public int Id { get; set; }
    public DateTime CreateDate { get; set; }
    public DateTime? ChangeDate { get; set; }
    public bool ViewState { get; set; } = true; // Soft Delete Flag
}
```

****مزایا****

- قابلیت بازیابی داده
- حفظ تاریخچه
- Audit Trail

۲.۴. Data Flow (جریان داده)

****جدید Task مثال: ایجاد یک****

```
User Request
↓
[Controller] TaskController.Create(model)
↓
[Validation] ModelState.IsValid?
↓
[Service] TaskService.CreateAsync(model)
↓
[Authorization] CanManageTask?
```

```

↓
[Business Logic] بررسی‌های کسب‌وکار
↓
[Repository] GenericRepository.AddAsync(task)
↓
[UnitOfWork] SaveChangesAsync()
↓
[Database] SQL Server Insert
↓
[Notification] NotificationService.CreateAsync()
↓
[SignalR] NotificationHub.SendNotification()
↓
[Response] Return to User

```

۲.۵. Real-time Architecture با SignalR

****Hub-based معماری****

```

Client Browser
↓ (WebSocket Connection)
[SignalR Hub] NotificationHub
↓
[Hub Groups] User-specific Groups
↓
[Broadcast] ارسال به کاربران خاص

```

****نحوه کارکرد****

1. برقرار می‌شود WebSocket کاربر وارد سیستم می‌شود → اتصال.
2. خودش اضافه می‌شود Group کاربر به `user\_{userId}`.
3. نوتیفیکیشن ایجاد می‌کند Service → (شد Task assign) وقتی اتفاقی می‌افتد.
4. NotificationHub به Group پیام می‌فرستد.
5. کاربر بلافاصله نوتیفیکیشن را دریافت می‌کند Browser.

۳. تکنولوژی‌ها و ابزارها - جزئیات فنی

۳.۱. ویژگی‌های استفاده شده - ASP.NET Core 8.0

ویژگی‌های استفاده شده - ASP.NET Core 8.0

استفاده شده MVC Pattern خیر، از **Minimal API**:

استفاده شده MVC Views خیر، از **Razor Pages**:

بله **MVC Pattern**:

برمی‌گردانند JSON ها Controller برخی **Web API**:

Development پشتیبانی در **Hot Reload**:

Program.cs پیکربندی‌های مهم در:

```
// Identity سفارشی
builder.Services.AddIdentity<ApplicationUser, IdentityRole<int>>(options =>
{
    options.Password.RequireDigit = true;
    options.Password.RequiredLength = 6;
    options.Password.RequireUppercase = false; // برای راحتی کاربران
    options.Password.RequireLowercase = false;
    options.Password.RequireNonAlphanumeric = false;
    options.User.RequireUniqueEmail = true;
});

// Cookie Authentication
builder.Services.ConfigureApplicationCookie(options =>
{
    options.LoginPath = "/Account/Login";
    options.AccessDeniedPath = "/Account/AccessDenied";
    options.Cookie.Name = "SmartTask";
    options.SlidingExpiration = true;
    options.ExpireTimeSpan = TimeSpan.FromDays(7);
});

// SignalR برای Real-time
builder.Services.AddSignalR();

// Session Management
builder.Services.AddSession(options =>
{
    options.IdleTimeout = TimeSpan.FromHours(2);
    options.Cookie.HttpOnly = true;
    options.Cookie.IsEssential = true;
});

// Background Services
builder.Services.AddHostedService<ReminderBackgroundService>();
```

```

builder.Services.AddHostedService<OverdueCascadeBackgroundService>();

// Authorization Policies
builder.Services.AddAuthorization(options =>
{
options.AddPolicy("AdminOnly", policy => policy.RequireRole("Admin"));
options.AddPolicy("ProjectManagerOnly", policy =>
policy.RequireRole("ProjectManager"));
options.AddPolicy("MemberOnly", policy => policy.RequireRole("Member"));
});

```

۳.۲. استفاده پیشرفته - Entity Framework Core 8.0

Code-First Approach:

- ها در کد تعریف می‌شوند Entity تمام
- Migration ها خودکار ایجاد می‌شوند
- اعمال می‌شود `dotnet ef migrations add` با Schema تغییرات

Fluent API Configurations:

پیکربندی جداگانه دارد `Data/Configurations/` در Entity هر

```

public class TaskItemConfiguration : IEntityTypeConfiguration<TaskItem>
{
public void Configure(EntityTypeBuilder<TaskItem> builder)
{
builder.ToTable("TaskItems");

builder.Property(x => x.Title)
.IsRequired()
.HasMaxLength(200);

builder.Property(x => x.Description)
.HasMaxLength(2000);

// Relationships
builder.HasOne(x => x.UserStory)
.WithMany(x => x.Tasks)
.HasForeignKey(x => x.UserStoryId)
.OnDelete(DeleteBehavior.Restrict); // جلوگیری از Cascade Delete

// Indexes for Performance
builder.HasIndex(x => x.Status);
builder.HasIndex(x => x.DueDate);
}
}

```

```
builder.HasIndex(x => new { x.UserStoryId, x.ViewState });
    }
}
```

****Query Optimization Techniques:****

```
// Eager Loading از جلوگیری از N+1 Query
var project = await _context.Projects
    .Include(x => x.Members)
    .ThenInclude(m => m.ApplicationUser)
    .Include(x => x.Sprints)
    .FirstOrDefaultAsync(x => x.Id == projectId);

// Projection برای بارگذاری فقط فیلدهای مورد نیاز
var tasks = await _context.TaskItems
    .Where(x => x.UserStory.ProjectId == projectId)
    .Select(x => new TaskListItemViewModel
    {
        Id = x.Id,
        Title = x.Title,
        Status = x.Status,
        AssigneeCount = x.Assignments.Count
    })
    .ToListAsync();

// AsNoTracking برای Read-Only Queries
var reports = await _context.Projects
    .AsNoTracking()
    .Where(x => x.WorkspaceId == workspaceId)
    .ToListAsync();
```

****ExecuteUpdateAsync و ExecuteDeleteAsync (EF Core 7+):****

```
// Bulk Update بدون Load کردن Entityها
await _context.Notifications
    .Where(x => x.ApplicationUserId == userId && !x.IsRead)
    .ExecuteUpdateAsync(setter => setter
        .SetProperty(n => n.IsRead, true)
        .SetProperty(n => n.ReadDate, DateTime.Now));

// Bulk Delete (Soft Delete)
await _context.TaskItems
    .Where(x => x.UserStoryId == storyId)
    .ExecuteUpdateAsync(setter => setter
        .SetProperty(t => t.ViewState, false));
```

****اعمال شده Indexes****

- Primary Key (Clustered Index) روی Id
- Non-Clustered Indexes روی Foreign Keys
- Composite Indexes روی Query های پرتکرار
- Index روی ViewState برای Soft Delete

****Query Performance****

- های پیچیده (در صورت نیاز Query برای Stored Procedures استفاده از
- Query Plan Caching توسط SQL Server
- Connection Pooling فعال

۳.۴. Real-time معماری - SignalR

****تعریف شده Hubs****

```
public class NotificationHub : Hub
{
    public override async Task OnConnectedAsync()
    {
        {
            var userId = Context.User?.FindFirst(ClaimTypes.NameIdentifier)?.Value;
            if (userId != null)
            {
                await Groups.AddToGroupAsync(Context.ConnectionId,
                    GetUserGroupName(int.Parse(userId)));
            }
            await base.OnConnectedAsync();
        }
    }

    public static string GetUserGroupName(int userId) => $"user_{userId}";
}
```

```
public class ChatHub : Hub
{
    {
        public async Task SendMessage(int projectId, string message)
        {
            await Clients.Group($"project_{projectId}").SendAsync("ReceiveMessage",
                message);
        }
    }
}
```

****Client-Side JavaScript****

```
const connection = new signalR.HubConnectionBuilder()
    .withUrl("/hubs/notification")
    .withAutomaticReconnect()
    .build();
```

```

connection.on("ReceiveNotification", (notification) => {
    showNotification(notification);
    updateBadgeCount();
});

connection.start();

```

کارهای پس‌زمینه - ۳.۵. Background Services

****ReminderBackgroundService:****

```

public class ReminderBackgroundService : BackgroundService
{
    protected override async Task ExecuteAsync(Cancellation_token stoppingToken)
    {
        while (!stoppingToken.IsCancellationRequested)
        {
            await CheckAndSendReminders();
            await Task.Delay(TimeSpan.FromMinutes(1), stoppingToken);
        }
    }

    private async Task CheckAndSendReminders()
    {
        var dueReminders = await _context.Reminders
            .Where(x => x.ReminderDate <= DateTime.Now && !x.IsSent)
            .ToListAsync();

        foreach (var reminder in dueReminders)
        {
            await _notificationService.CreateAsync(
                reminder.ApplicationUserId,
                reminder.Title,
                $"یادآور: {reminder.TaskItem.Title}",
                NotificationType.Reminder
            );
            reminder.IsSent = true;
        }

        await _context.SaveChangesAsync();
    }
}

```

****OverdueCascadeBackgroundService:****

```

// های سررسید گذشته را بررسی می‌کند Task هر شب ساعت 2 بامداد
// وضعیت آن‌ها را آپدیت می‌کند

```

بخش دوم: موجودیت‌ها و مدل داده

۴. موجودیت‌های اصلی (Core Entities)

را پیاده‌سازی می‌کنند Agile/Scrum این موجودیت‌ها ستون فقرات سیستم را تشکیل می‌دهند و ساختار اصلی

۴.۱. ApplicationUser (کاربر)

مسئولیت **:نگهداری اطلاعات کاربران سیستم**

(ASP.NET Core Identity) ارث می‌برد `IdentityUser<int>` ارث‌بری **:از**

****فیلدهای اضافی****

```
public class ApplicationUser : IdentityUser<int>
{
    // اطلاعات پایه
    // UserName, Email, PasswordHash, PhoneNumber, ...

    // فیلدهای سفارشی
    public string FullName { get; set; }
    public string? Avatar { get; set; }
    public string? Bio { get; set; }
    public DateTime CreatedDate { get; set; }

    // Navigation Properties
    public ICollection<WorkspaceMember> WorkspaceMemberships { get; set; }
    public ICollection<ProjectMember> ProjectMemberships { get; set; }
    public ICollection<TaskAssignment> TaskAssignments { get; set; }
    public ICollection<Comment> Comments { get; set; }
    public ICollection<TimeLog> TimeLogs { get; set; }
    public ICollection<Notification> Notifications { get; set; }
}
```

****نقش‌های سیستمی****

- دسترسی کامل به سیستم: `Admin`
- کاربر عادی: `User`

۴.۲. Workspace (فضای کاری)

Slack در "Workspace" یا GitHub در "Organization" مسئولیت ****بالاترین سطح سازمان‌دهی. مشابه****

****Entity Class.****

```
public class Workspace
{
    public int Id { get; set; }
    public string Name { get; set; } // نام Workspace
    public string? Description { get; set; }
    public string? Logo { get; set; } // مسیر لوگو
    public string? Color { get; set; } // رنگ تم
    public string? TimeZone { get; set; } // پیش‌فرض: منطقه زمانی (Iran)

    public VisibilityType Visibility { get; set; } // Public/Private
    public bool IsActive { get; set; }
    public DateTime CreateDate { get; set; }
    public bool ViewState { get; set; } // Soft Delete

    // Owner
    public int OwnerId { get; set; }
    public ApplicationUser Owner { get; set; }

    // Collections
    public ICollection<Project> Projects { get; set; }
    public ICollection<Team> Teams { get; set; }
    public ICollection<WorkspaceMember> Members { get; set; }
}
```

****Workspace نقش‌های****

- مالک (کامل‌ترین دسترسی): `Owner`
- مدیر (دسترسی مدیریتی): `Admin`
- مدیر پروژه: `ProjectManager`
- توسعه‌دهنده: `Developer`
- تستر: `Tester`
- مشاهدگر (فقط خواندن): `Viewer`

****Use Cases:****

- Workspace یک شرکت = یک
- Workspace یک استارتاپ = یک
- Workspace یک دپارتمان = یک

۴.۳. Project (پروژه)

است Task و UserStory، Backlog، Sprint مسئولیت ****پروژه‌ها محل اصلی کار هستند. هر پروژه شامل****

****Entity Class:****

```
public class Project : BaseEntity
{
    // Basic Info
    public int WorkspaceId { get; set; }
    public string Name { get; set; }
    public string Key { get; set; } // کلید یکتا (مثل PROJ-123)
    public string? Description { get; set; }
    public string? Color { get; set; }
    public string? Icon { get; set; }

    // Scheduling
    public DateTime? StartDate { get; set; }
    public DateTime? DueDate { get; set; }
    public DateTime? EndDate { get; set; } // تاریخ پایان واقعی

    // Status & Priority
    public ProjectStatusType Status { get; set; }
    // Planning, InProgress, OnHold, Completed, Cancelled
    public ProjectPriorityType Priority { get; set; }
    // Low, Medium, High, Critical

    public bool IsArchived { get; set; } // پروژه آرشیو شده

    // Relationships
    public Workspace Workspace { get; set; }
    public ICollection<ProjectMember> Members { get; set; }
    public ICollection<Sprint> Sprints { get; set; }
    public ICollection<UserStory> UserStories { get; set; }
    public Backlog? Backlog { get; set; } // هر پروژه یک
    public ICollection<Label> Labels { get; set; }
    public ICollection<ProjectTeam> ProjectTeams { get; set; }
}
```

****ویژگی‌های کلیدی****

- ****Key****: شناسه کوتاه و یکتا ("SMART", "PROJ")
- ****Status Lifecycle****: Planning → InProgress → Completed/Cancelled
- ****Archive****: پروژه‌های قدیمی بدون حذف
- ****Color & Icon****: شخصی‌سازی ظاهر

****Project نقش‌های****

- **`Manager`**: مدیر پروژه (دسترسی کامل)
- **`Developer`**: توسعه‌دهنده
- **`Tester`**: تستر
- **`Viewer`**: مشاهدگر

۴.۴. اسپرینت (اسپرینت)

مسئولیت ****بازه زمانی ثابت (معمولاً ۲-۴ هفته (برای تحویل قابلیت‌های کاری****

****Entity Class****

```
public class Sprint : BaseEntity
{
    public int ProjectId { get; set; }
    public string Name { get; set; } // مثل "Sprint 1", "Release 2.0"
    public string? Goal { get; set; } // هدف Sprint

    public DateTime StartDate { get; set; }
    public DateTime EndDate { get; set; }

    public int Capacity { get; set; } // ظرفیت به ساعت

    public SprintStatusType Status { get; set; }
    // Planning, Active, Completed, Cancelled

    // Relationships
    public Project Project { get; set; }
    public ICollection<UserStory> UserStories { get; set; }
}
```

****محدودیت‌های کسب‌وکار****

- باشد `Active` می‌تواند Sprint فقط یک
- ها نباید تداخل داشته باشند Sprint تاریخ
- معمولاً (=تعداد اعضا × ساعات کاری ×تعداد روزها) Capacity

****Sprint Planning Process:****

1. Product Owner اولویت‌بندی Backlog را انجام می‌دهد

2. منتقل می‌کند Sprint به Backlog ها را از UserStory تیم

3. Sprint `Active` می‌شود

4. ها را انجام می‌دهد Task تیم

5. Sprint `Completed` می‌شود

6. Sprint Review و Retrospective

۴.۵. Backlog (بک‌لاگ)

اختصاص نیافته‌اند Sprint هایی که هنوز به UserStory مسئولیت ****لیست اولویت‌دار****

****Entity Class:****

```
public class Backlog : BaseEntity
{
    public int ProjectId { get; set; }
    public string Name { get; set; } // معمولاً "Product Backlog"
    public string? Description { get; set; }

    // Relationships
    public Project Project { get; set; }
    public ICollection<UserStory> UserStories { get; set; }
}
```

دارد که خودکار ایجاد می‌شود Backlog فقط یک Project نکته ****هر****

۴.۶. UserStory (داستان کاربری)

Scrum. مسئولیت **نیازمندی از دیدگاه کاربر. واحد کار در**

****Entity Class:****

```
public class UserStory : BaseEntity
{
    // Foreign Keys
    public int ProjectId { get; set; }
    public int BacklogId { get; set; }
    public int? SprintId { get; set; } // nullable: در Backlog یا Sprint
    public int? OwnerId { get; set; } // مسئول Story

    // Basic Info
    public string Title { get; set; }
    public string? Description { get; set; }
    public string? AcceptanceCriteria { get; set; } // معیارهای پذیرش

    // Planning
    public int StoryPoint { get; set; } // تخمین پیچیدگی
    public int BusinessValue { get; set; } // ارزش تجاری
    public int Order { get; set; } // Backlog ترتیب در

    public StoryPriorityType Priority { get; set; }
    // Low, Medium, High, Critical
    public StoryStatusType Status { get; set; }
    // New, InProgress, Testing, Done

    // Relationships
    public Project Project { get; set; }
    public Backlog Backlog { get; set; }
    public Sprint? Sprint { get; set; }
    public ApplicationUser? Owner { get; set; }
    public ICollection<TaskItem> Tasks { get; set; }
}
```

****Story Points:**** (معمولاً) واحد تخمین پیچیدگی (Fibonacci: 1, 2, 3, 5, 8, 13, 21)

****Acceptance Criteria:**** باید برآورده کند Story شرایطی که

به عنوان یک [کاربر]
می‌خواهم [قابلیت]
تا بتوانم [هدف]

معیارهای پذیرش:

- شرط 1
- شرط 2
- شرط 3

****Sprint و Backlog حرکت بین****

```
// انتقال به Sprint
await _userService.MoveToSprintAsync(storyId, sprintId);

// بازگشت به Backlog
await _userService.RemoveFromSprintAsync(storyId);
```

۴.۷. TaskItem (وظیفه)

انجام می‌شود Task با چندین UserStory مسئولیت ****کوچکترین واحد کاری قابل تخصیص پیاده‌سازی یک****

****Entity Class.****

```
public class TaskItem : BaseEntity
{
    public int UserStoryId { get; set; }

    // Basic Info
    public string Title { get; set; }
    public string? Description { get; set; }

    // Type & Status
    public TaskType Type { get; set; }
    // Task, Bug, Feature, Improvement
    public TaskStatusType Status { get; set; }
    // ToDo, InProgress, InReview, Done, Cancelled
    public TaskPriorityType Priority { get; set; }
    // Low, Medium, High, Critical

    // Time Management
    public int Estimate { get; set; } // تخمین زمان (ساعت)
    public DateTime? StartDate { get; set; }
    public DateTime? DueDate { get; set; }
    public DateTime? CompletedDate { get; set; }

    // Relationships
    public UserStory UserStory { get; set; }
    public ICollection<SubTaskItem> SubTasks { get; set; }
    public ICollection<TaskAssignment> Assignments { get; set; }
    public ICollection<Comment> Comments { get; set; }
    public ICollection<Attachment> Attachments { get; set; }
    public ICollection<Checklist> Checklists { get; set; }
```

```

public ICollection<TaskLabel> TaskLabels { get; set; }
public ICollection<Reminder> Reminders { get; set; }
public ICollection<ActivityLog> ActivityLogs { get; set; }
public ICollection<TimeLog> TimeLogs { get; set; }
public ICollection<TaskDependency> Dependencies { get; set; }
}

```

****Task Types.****

- ****Task****: کار عادی
- ****Bug****: رفع اشکال
- ****Feature****: قابلیت جدید
- ****Improvement****: بهبود

۳.۱. Workspace (فضای کاری)

Project، است و می‌تواند شامل چندین Owner متعلق به یک Workspace بالاترین سطح سازمان‌دهی در سیستم. هر Team و Member باشد.

****ویژگی‌های کلیدی****

- نام، توضیحات، لوگو و رنگ سفارشی
- (TimeZone) مدیریت منطقه زمانی
- (Public/Private) سطوح دسترسی
- (Owner, Admin, ProjectManager, Developer, Tester, Viewer) مدیریت اعضا با نقش‌های مختلف

۳.۲. Project (پروژه)

قرار می‌گیرند و محل اصلی برنامه‌ریزی و اجرای کار هستند Workspace پروژه‌ها در داخل

****ویژگی‌های کلیدی****

- یکتا، توضیحات، رنگ و آیکون Key، نام
- تاریخ شروع، سررسید و پایان
- (Planning, InProgress, OnHold, Completed, Cancelled) وضعیت
- (Low, Medium, High, Critical) اولویت
- قابلیت آرشیو کردن
- (Manager, Developer, Tester, Viewer) مدیریت اعضای پروژه با نقش‌ها

- مدیریت تیم‌های اختصاص یافته

۳.۳. Sprint (اسپرینت)

UserStory بازه زمانی مشخص برای انجام مجموعه‌ای از

****ویژگی‌های کلیدی****

- نام، هدف و تاریخ شروع و پایان
- به ساعت (Capacity) ظرفیت
- (Planning, Active, Completed, Cancelled) وضعیت
- فعال باشد Project می‌تواند در هر Sprint فقط یک
- جلوگیری از تداخل تاریخ اسپرینت‌ها

۳.۴. UserStory (داستان کاربری)

قرار گیرد Sprint یا Backlog نیازمندی یا ویژگی از دید کاربر که می‌تواند در

****ویژگی‌های کلیدی****

- عنوان و توضیحات
- (ToDo, InProgress, Testing, Done) وضعیت
- (Low, Medium, High, Critical) اولویت
- Story Point تخمین
- مالک (Owner)
- Order مرتب‌سازی با
- Sprint و Backlog قابل جابجایی بین

۳.۵. TaskItem (وظیفه)

متصل است UserStory کوچکترین واحد کاری قابل تخصیص که به یک

****ویژگی‌های کلیدی****

- (Task, Bug, Feature, Improvement) عنوان، توضیحات و نوع
- (ToDo, InProgress, InReview, Done, Cancelled) وضعیت

- اولویت (Low, Medium, High, Critical)
- به ساعت (Estimate) تخمین زمانی
- تاریخ شروع، سررسید و تکمیل
- قابلیت تخصیص به چندین نفر
- SubTask، Checklist، Comment، Attachment
- (Label) برچسب‌گذاری
- (TimeLog) ثبت زمان کار
- (Reminder) یادآورها
- (Dependencies) وابستگی‌ها

قابلیت‌های اصلی سیستم ۴.

مدیریت کاربران و دسترسی‌ها ۴.۱.

- Identity ثبت‌نام و ورود با
- احراز هویت دو مرحله‌ای
- (قابل فعال‌سازی) Google ورود با
- Admin, User: نقش‌های سیستمی
- Workspace: Owner, Admin, ProjectManager, Developer, Tester, Viewer نقش‌های
- Project: Manager, Developer, Tester, Viewer نقش‌های
- مدیریت پروفایل و تنظیمات کاربری

داشبوردها و گزارشات ۴.۲.

- کاربر: **Dashboard** نمای کلی وظایف و فعالیت‌ها
- آمار پروژه‌ها و اعضا: **Dashboard Workspace**
- KPI پیشرفت، نمودارها و: **Dashboard Project**
- آمار کل سیستم: **Dashboard Admin**
- Workspace و Sprint، گزارشات تحلیلی پروژه
- Velocity و Burndown نمودار
- (Health) تحلیل سلامت پروژه
- (Delay Risk) ریسک تاخیر
- (Workload Analysis) تحلیل بار کاری

تابلوها و نماها ۴.۳.

- **Task Board**: Drag & Drop با Kanban نمای
- **Backlog**: مدیریت UserStory های بدون
- **Sprint Planning**: Sprint و Backlog بین Story جابجایی
- فیلترهای پیشرفته: وضعیت، اولویت، تخصیص، برچسب
- Timeline (Gantt-like) نمای
- نمای تقویم

همکاری و ارتباطات ۴.۴.

- Task بر روی (Comments) نظرات
- (@username) منشن کردن اعضا
- SignalR با Real-time نوتیفیکیشن
- Activity Log برای تمام تغییرات
- AI Assistant (LM Studio) چت گروهی با
- Workspace ارسال دعوتنامه به

مدیریت زمان ۴.۵.

- Task بر روی (TimeLog) ثبت زمان کار
- تایمر شروع/توقف
- (Reminder) یادآورهای زمانی
- Task گزارش زمان صرف شده به ازای کاربر، پروژه و
- مقایسه زمان تخمینی و واقعی

ویژگی‌های پیشرفته ۴.۶.

- Task (Dependencies): Blocked By, Blocks وابستگی‌های
- Task Trade (Task Trade) بین Task مبادله
- AI با کمک Task تقسیم: **Task Breakdown**
- (Priority Engine) تحلیل اولویت هوشمند
- وظایف خارج از پروژه: **Offroad Task**

- (Labels) **برچسب‌گذاری رنگی**
- Task در **Checklist**
- **SubTask**
- (Attachments) **ضمیمه فایل**

۵. سرویس‌های سیستم

:شامل ۴۵ +سرویس است که منطق کسب‌وکار را پیاده‌سازی می‌کنند Service لایه

۶. ساختار پایگاه داده

.مدیریت می‌شوند Entity Framework Core پایگاه داده شامل ۲۵ +جدول اصلی است که با

۶.۱. جداول اصلی

- کاربران سیستم:AspNetUsers
- فضا‌های کاری:Workspaces
- پروژه‌ها:Projects
- اسپرینت‌ها:Sprints
- بک‌لاگ‌ها:Backlogs
- داستان‌های کاربری:UserStories
- وظایف:TaskItems
- تخصیص وظایف:TaskAssignments
- زیروظایف:SubTaskItems
- نظرات:Comments
- فایل‌های ضمیمه:Attachments
- چک‌لیست‌ها:Checklists & ChecklistItems
- برچسب‌ها:Labels & TaskLabels
- ثبت زمان:TimeLogs
- یادآور‌ها:Reminders
- وابستگی‌ها:TaskDependencies
- لاگ فعالیت‌ها:ActivityLogs
- نوتیفیکیشن‌ها:Notifications
- تیم‌ها:Teams & TeamMembers
- Workspace اعضای:WorkspaceMembers
- اعضای پروژه:ProjectMembers
- دعوت‌نامه‌ها:WorkspaceInvitations

- OffroadTasks: وظایف خارج از پروژه

روابط کلیدی ۶.۲.

- Workspace 1→N Project
- Project 1→N Sprint
- Project 1→1 Backlog
- Project 1→N UserStory
- Sprint 1→N UserStory (optional)
- UserStory 1→N TaskItem
- TaskItem N→N ApplicationUser (via TaskAssignment)
- TaskItem 1→N Comment
- TaskItem 1→N Attachment
- TaskItem 1→N TimeLog
- TaskItem 1→N SubTaskItem
- TaskItem N→N Label (via TaskLabel)
- TaskItem N→N TaskItem (via TaskDependency)

کنترلرهای سیستم ۷.

است: MVC سیستم شامل ۴۰+کنترلر

پیکربندی و تنظیمات ۸.

اتصال به دیتابیس ۸.۱.

تعریف شده است. سه محیط موجود SQL Server کانکشن استرینگ appsettings.json در فایل

- محلی (MAEDE)
- (سرور راه دور) 92.246.145.99)
- (DESKTOP-L1ICADA\BEHDADSERVER) سرور محلی دیگر

سرویس ایمیل ۸.۲.

- برای ارسال ایمیل SMTP Gmail
- Host: smtp.gmail.com
- Port: 587
- SSL فعال

۸.۳. LM Studio (AI)

:استفاده می‌کند Task برای چت و تجزیه (Google Gemma) محلی AI سیستم از یک مدل

- Model: google/gemma-4-12b
- BaseUrl: http://92.246.145.99:1234/v1/chat/completions

۸.۴. Webpushr

.برای اطلاع‌رسانی خارج از مرورگر Push Notification سرویس

۹. امنیت و دسترسی

۹.۱. احراز هویت

- برای مدیریت کاربران ASP.NET Core Identity
- Identity پسورد هش شده با استانداردهای
- (2FA) احراز هویت دو عاملی
- Google قابلیت ورود با

۹.۲. مجوزها

- Task و Project Workspace بررسی دسترسی در سطح
- CanManageWorkspace, CanManageProject, سرویس‌های مجوز CanManageBacklog
- (Role-based) نقش‌محور
- (Row-level security) کنترل دسترسی به سطرها

۹.۳. حذف منطقی (Soft Delete)

هستند که به جای حذف فیزیکی، رکوردها را غیرفعال می‌کند ViewState تمام موجودیت‌ها دارای فیلد

تست‌ها ۱۰.

است که سرویس‌های اصلی را پوشش می‌دهد Moq و XUnit هایی با Unit Test پروژه شامل

- TaskServiceTests
- UserStoryServiceTests
- SprintServiceTests
- SubTaskServiceTests
- TaskDependencyServiceTests
- WorkloadAnalysisServiceTests
- ProjectHealthServiceTests
- PriorityEngineServiceTests
- DateFormatServiceTests

استفاده می‌کنند و نیازی به دیتابیس واقعی ندارند InMemory Database تست‌ها از

۱۱. استقرار (Deployment)

نیازمندی‌های سرور ۱۱.۱.

- .NET 8.0 Runtime
- یا بالاتر SQL Server 2019
- Kestrel یا IIS
- GB RAM حداقل ۴
- فضای دیسک ۱۰ GB

مراحل استقرار ۱۱.۲.

1. Publish با dotnet publish پروژه
2. کپی فایل‌ها به سرور
3. تنظیم Connection String
4. Migration اجرای
5. Kestrel یا IIS پیکربندی
6. تنظیم SSL Certificate
7. Background Services راه‌اندازی
8. SignalR Connection بررسی

(Roadmap) ویژگی‌های آینده ۱۲.

- Mobile App (React Native / Flutter)
- Git (GitHub, GitLab) اینتگریشن با
- تقویم تعاملی پیشرفته
- کامل Gantt Chart
- Resource Management
- Budget & Cost Tracking
- Slack/Teams اینتگریشن با
- Custom Workflows
- Advanced Analytics & BI
- Multi-language Support
- Dark Mode
- API Documentation (Swagger)
- Webhooks

پیوست: اصطلاحات کلیدی

****پایان مستند****

است SmartTask این مستند خلاصه‌ای از ساختار، معماری و ویژگی‌های سیستم

برای اطلاعات بیشتر به کد منبع پروژه مراجعه کنید